

# 校园开放预约与访客管理系统 — 课程设计报告

课程名称：数据库系统课程设计  
小组成员：范伟程、韩中信、崔鲔泮  
完成日期：2026年6月

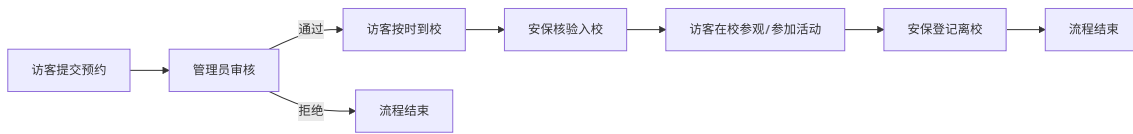
## 目录

- 项目概述
- 需求分析
- 技术框架选取
- 系统架构设计
- 数据库设计（重点）
  - 5.1 概念结构设计（ER图）
  - 5.2 逻辑结构设计（数据字典）
  - 5.3 物理结构设计（约束、索引、触发器、函数）
- 后端设计
- 前端设计
- 成果展示与功能演示
- 功能与需求对照表
- 前端-后端-数据库关联分析
- 总结与展望

## 1. 项目概述

本系统为**校园一体化访客管理数据库系统**，整合了校园开放预约、访客权限管控、校园活动报名、客流容量控制、安全审计追溯等核心功能。系统面向**校外访客**、**校园管理员**、**安保人员**、**志愿工作人员**四类角色，实现访客入校全流程数字化、规范化、智能化管理。

系统核心业务流程为：



## 2. 需求分析

### 2.1 用户角色分析

| 角色                 | 职责        | 核心需求                                       |
|--------------------|-----------|--------------------------------------------|
| 校外访客<br>(visitor)  | 预约入校、报名活动 | 提交预约、查看审核状态、浏览活动、取消预约                      |
| 管理员 (admin)        | 全局管理与配置   | 审核预约、管理开放规则、管理区域/校门、活动管理、黑名单管理、排班管理、查看数据看板 |
| 安保人员<br>(security) | 校门核验      | 核验预约信息、登记入校/离校、查看当前在校访客                    |
| 工作人员 (staff)       | 志愿引导与讲解   | 查看排班、配合活动执行                                |

## 2.2 功能需求清单

| 编号     | 功能模块    | 需求描述                                 | 优先级 |
|--------|---------|--------------------------------------|-----|
| REQ-01 | 用户注册/登录 | 访客通过手机号注册，所有角色通过手机号+密码登录，系统使用 JWT 认证 | 高   |
| REQ-02 | 入校预约    | 访客选择日期、时段、访客类型，填写个人信息和入校事由，提交预约申请    | 高   |
| REQ-03 | 预约审核    | 管理员查看待审核预约列表，审批通过或拒绝，填写审核意见          | 高   |
| REQ-04 | 入校核验    | 安保人员通过预约编号/手机号/身份证号查询预约，确认后登记入校      | 高   |
| REQ-05 | 离校登记    | 安保人员登记访客离校，记录离校时间和校门                 | 高   |
| REQ-06 | 校园活动管理  | 管理员发布/编辑/删除活动，设置报名上限；访客浏览活动并报名       | 中   |
| REQ-07 | 活动签到    | 管理员为已报名访客进行现场签到                      | 中   |
| REQ-08 | 开放规则配置  | 管理员设置不同日期类型的开放时段和容量上限                | 高   |
| REQ-09 | 区域权限管理  | 定义校园区域，配置不同访客类型的可进入权限                | 中   |
| REQ-10 | 黑名单管理   | 管理员将违规访客加入黑名单，设置拉黑期限                 | 中   |
| REQ-11 | 违规举报    | 用户举报校园内的违规行为，管理员审核处理                 | 低   |
| REQ-12 | 排班管理    | 管理员为安保和工作人员排班                        | 低   |
| REQ-13 | 数据看板    | 管理员查看今日预约、在校人数、待审核数等统计               | 中   |
| REQ-14 | 审计日志    | 全量记录所有管理操作，实现全程可追溯                   | 中   |
| REQ-15 | 状态流转追溯  | 记录预约每一次状态变更，从创建到离校全链路可查              | 高   |

## 2.3 非功能需求

- **安全性**：密码 BCrypt 哈希存储，JWT 令牌认证，基于角色的访问控制（RBAC）
- **数据完整性**：外键约束、CHECK 约束、唯一约束保证数据一致性
- **可追溯性**：预约状态变更触发器自动记录日志，审计日志记录所有后台操作
- **可扩展性**：分层架构（Controller → Service → Repository/EF Core），便于后续扩展

## 3. 技术框架选取

### 3.1 技术选型理由

| 层级       | 技术                    | 版本       | 选型理由                                                          |
|----------|-----------------------|----------|---------------------------------------------------------------|
| 数据库      | SQL Server            | 2019+    | 课程要求使用关系型数据库，SQL Server 提供完善的约束、触发器、存储过程支持，适合展示数据库设计能力        |
| 后端框架     | ASP.NET Core Web API  | .NET 8.0 | 微软官方框架，内置依赖注入、JWT 认证中间件，Entity Framework Core 作为 ORM 实现代码优先开发 |
| ORM      | Entity Framework Core | 8.0      | 与 .NET 深度集成，支持 Fluent API 配置关系映射，自动迁移，LINQ 查询语法简洁             |
| 前端框架     | Vue 3                 | 3.x      | 渐进式框架，组合式 API（Composition API）使逻辑复用更简便，学习曲线平缓                 |
| UI 组件库   | Element Plus          | 2.x      | 成熟的 Vue 3 组件库，提供丰富的表单、表格、对话框等组件，适合后台管理系统                      |
| 构建工具     | Vite                  | 5.x      | 新一代前端构建工具，开发服务器秒级启动，HMR 热更新体验极佳                               |
| HTTP 客户端 | Axios                 | 1.x      | 基于 Promise 的 HTTP 库，支持请求/响应拦截器，便于统一处理认证 Token                 |
| 路由       | Vue Router            | 4.x      | Vue 官方路由，支持嵌套路由和路由守卫，实现基于角色的页面访问控制                            |
| 认证       | JWT (Bearer Token)    | —        | 无状态认证方案，适合前后端分离架构，BCrypt 加密密码                                 |

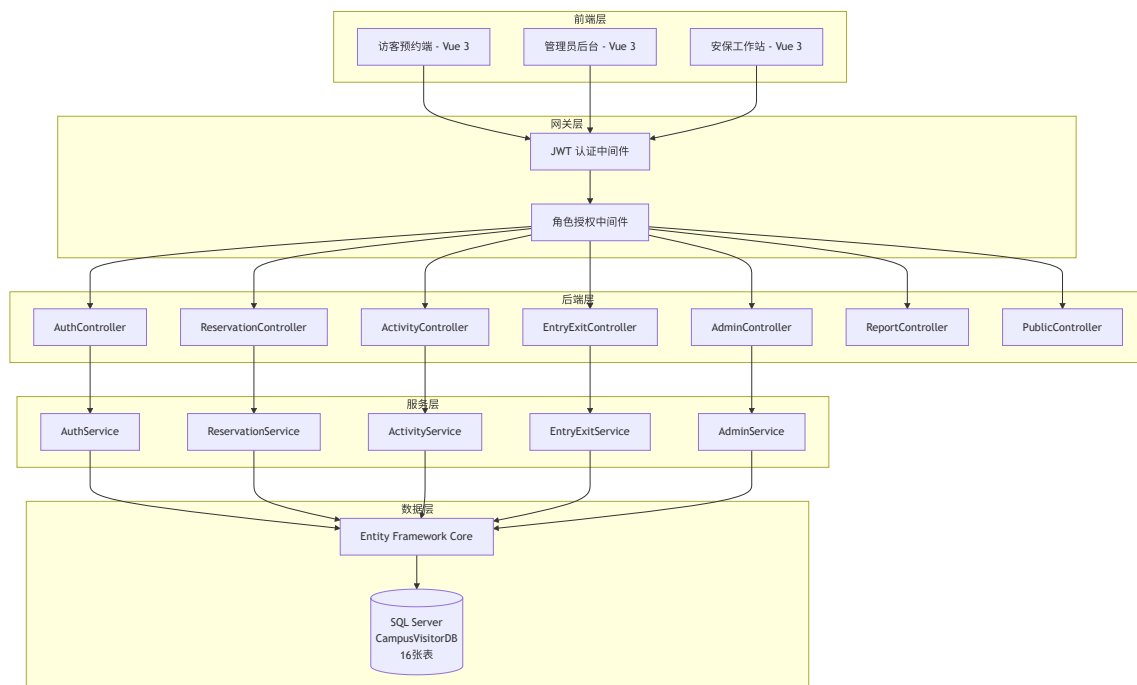
### 3.2 架构选型理由

采用前后端分离架构的核心考量：

- 1. 开发解耦：前端和后端可以独立开发和调试，互不阻塞
- 2. 多端复用：后端 API 可同时服务 Web 端、移动端等不同前端
- 3. 职责清晰：后端专注业务逻辑和数据访问，前端专注用户交互和界面展示
- 4. 符合行业趋势：前后端分离是当前 Web 开发的主流模式

## 4. 系统架构设计

### 4.1 总体架构



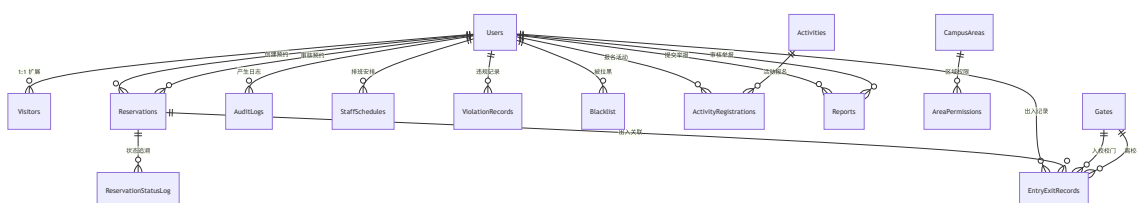
## 4.2 分层职责

| 层级         | 职责                                    | 实现方式                                         |
|------------|---------------------------------------|----------------------------------------------|
| Controller | 接收 HTTP 请求，参数校验，调用 Service，返回 HTTP 响应 | ASP.NET Core Controller + [ApiController] 特性 |
| Service    | 核心业务逻辑，事务管理，调用 EF Core 进行数据操作         | 接口+实现分离 (IAuthService / AuthService)         |
| DbContext  | 数据库上下文，定义实体映射关系，管理数据库连接               | CampusVisitorDbContext : DbContext           |
| Model      | 数据库实体映射，定义表结构、字段约束、导航属性               | C# POCO 类 + Data Annotations + Fluent API    |

## 5. 数据库设计

数据库名称：CampusVisitorDB  
数据库引擎：SQL Server 2019  
总表数量：16 张  
设计原则：第三范式（3NF），合理的冗余以优化查询性能

### 5.1 概念结构设计（ER 图）



ER 图说明：

- Users** 是系统的**中心实体**，统一管理所有角色（访客、管理员、安保、工作人员），通过 **Role** 字段区分
- Visitors** 是 **Users** 的 **1:1 扩展表**，存放访客特有信息（身份证号、访客类型等）
- Reservations** 是**核心业务表**，承载预约申请、审核、入校、离校全流程
- ReservationStatusLog** 是**审计关联表**，通过触发器自动记录每一次状态变更

- `CampusAreas` 和 `AreaPermissions` 构成多对多关系，实现不同访客类型对不同区域的差异化访问权限
- `EntryExitRecords` 同时关联 `Reservations`、`Users`、`Gates`（入口/出口），形成完整的出入轨迹

## 5.2 逻辑结构设计（数据字典）

以下是全部 16 张表的数据字典：

### 5.2.1 Users（用户表） — 统一身份管理

| 字段名                       | 数据类型         | 约束                                                                                  | 说明          |
|---------------------------|--------------|-------------------------------------------------------------------------------------|-------------|
| <code>Id</code>           | INT          | <b>PK</b> , IDENTITY(1,1)                                                           | 用户唯一标识      |
| <code>Name</code>         | NVARCHAR(50) | NOT NULL                                                                            | 用户姓名        |
| <code>Phone</code>        | VARCHAR(20)  | NOT NULL, <b>UNIQUE</b>                                                             | 手机号（登录账号）   |
| <code>PasswordHash</code> | VARCHAR(256) | NOT NULL                                                                            | BCrypt 密码哈希 |
| <code>Role</code>         | VARCHAR(20)  | NOT NULL, DEFAULT 'visitor',<br><b>CHECK</b> ('visitor','admin','security','staff') | 用户角色        |
| <code>Email</code>        | VARCHAR(100) | NULL                                                                                | 邮箱          |
| <code>IsActive</code>     | BIT          | NOT NULL, DEFAULT 1                                                                 | 账户是否激活      |
| <code>CreatedAt</code>    | DATETIME2    | NOT NULL, DEFAULT GETDATE()                                                         | 注册时间        |
| <code>UpdatedAt</code>    | DATETIME2    | NOT NULL, DEFAULT GETDATE()                                                         | 更新时间        |
| <code>LastLoginAt</code>  | DATETIME2    | NULL                                                                                | 最后登录时间      |

### 5.2.2 Visitors（访客信息扩展表）

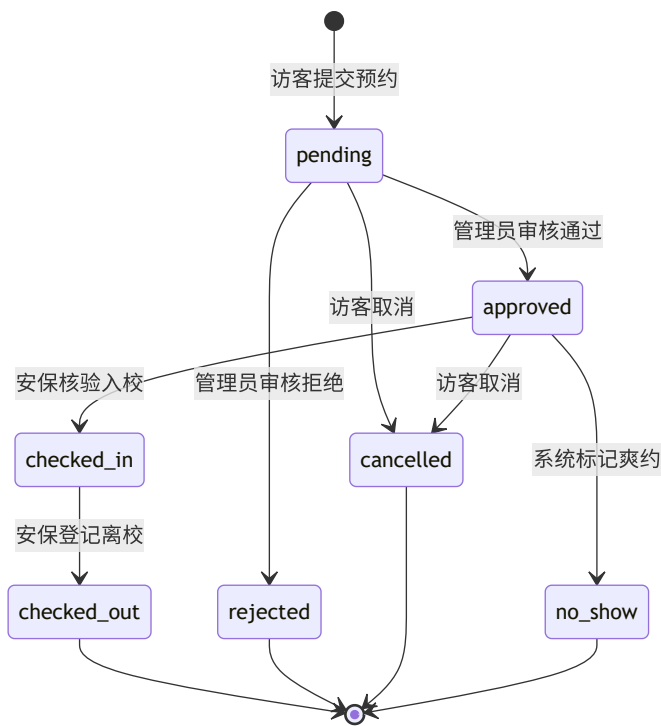
| 字段名                      | 数据类型          | 约束                                | 说明    |
|--------------------------|---------------|-----------------------------------|-------|
| <code>Id</code>          | INT           | <b>PK</b> , IDENTITY(1,1)         | 记录 ID |
| <code>UserId</code>      | INT           | NOT NULL, <b>FK</b> → Users(Id)   | 关联用户  |
| <code>IdCard</code>      | VARCHAR(20)   | NULL                              | 身份证号  |
| <code>Gender</code>      | VARCHAR(4)    | NULL, <b>CHECK</b> ('男','女','其他') | 性别    |
| <code>BirthDate</code>   | DATE          | NULL                              | 出生日期  |
| <code>Affiliation</code> | NVARCHAR(100) | NULL                              | 所属    |

| 字段名              | 数据类型          | 约束                                                                                                 | 说明     |
|------------------|---------------|----------------------------------------------------------------------------------------------------|--------|
|                  |               |                                                                                                    | 单位     |
| VisitorType      | VARCHAR(20)   | NOT NULL, DEFAULT 'tourist',<br><b>CHECK</b> ('parent','alumni','tourist','study_group','partner') | 访客类型   |
| EmergencyContact | VARCHAR(20)   | NULL                                                                                               | 紧急联系人  |
| EmergencyPhone   | VARCHAR(20)   | NULL                                                                                               | 紧急联系电话 |
| Remarks          | NVARCHAR(500) | NULL                                                                                               | 备注     |

5.2.3 Reservations（预约申请表）— 核心表

| 字段名           | 数据类型          | 约束                                                                                                                          |
|---------------|---------------|-----------------------------------------------------------------------------------------------------------------------------|
| Id            | INT           | <b>PK</b> , IDENTITY(1,1)                                                                                                   |
| UserId        | INT           | NOT NULL, <b>FK</b> → Users(Id)                                                                                             |
| ReservationNo | VARCHAR(30)   | NOT NULL, <b>UNIQUE</b>                                                                                                     |
| VisitorType   | VARCHAR(20)   | NOT NULL, <b>CHECK</b> ('parent','alumni','tourist','study_group','partner')                                                |
| VisitorName   | NVARCHAR(50)  | NOT NULL                                                                                                                    |
| VisitorPhone  | VARCHAR(20)   | NOT NULL                                                                                                                    |
| VisitDate     | DATE          | NOT NULL                                                                                                                    |
| TimeSlot      | VARCHAR(20)   | NOT NULL, <b>CHECK</b> ('morning','afternoon','full_day')                                                                   |
| Companions    | INT           | NOT NULL, DEFAULT 0                                                                                                         |
| StayDuration  | VARCHAR(20)   | NULL                                                                                                                        |
| Purpose       | NVARCHAR(500) | NOT NULL                                                                                                                    |
| Status        | VARCHAR(20)   | NOT NULL, DEFAULT 'pending',<br><b>CHECK</b> ('pending','approved','rejected','cancelled','checked_in','checked_out','no_sh |
| ReviewerId    | INT           | NULL, <b>FK</b> → Users(Id)                                                                                                 |
| ReviewRemark  | NVARCHAR(500) | NULL                                                                                                                        |
| ReviewedAt    | DATETIME2     | NULL                                                                                                                        |
| CreatedAt     | DATETIME2     | NOT NULL, DEFAULT GETDATE()                                                                                                 |
| UpdatedAt     | DATETIME2     | NOT NULL, DEFAULT GETDATE()                                                                                                 |

状态流转逻辑：



5.2.4 ReservationStatusLog（预约状态变更日志表）

| 字段名           | 数据类型          | 约束                              | 说明    |
|---------------|---------------|---------------------------------|-------|
| Id            | INT           | PK, IDENTITY(1,1)               | 日志ID  |
| ReservationId | INT           | NOT NULL, FK → Reservations(Id) | 关联预约  |
| FromStatus    | VARCHAR(20)   | NULL                            | 变更前状态 |
| ToStatus      | VARCHAR(20)   | NOT NULL                        | 变更后状态 |
| OperatorId    | INT           | NULL, FK → Users(Id)            | 操作人   |
| Remark        | NVARCHAR(500) | NULL                            | 备注说明  |
| CreatedAt     | DATETIME2     | NOT NULL, DEFAULT GETDATE()     | 记录时间  |

5.2.5 CampusAreas（校园区域表）

| 字段名         | 数据类型          | 约束                                                                                             | 说明   |
|-------------|---------------|------------------------------------------------------------------------------------------------|------|
| Id          | INT           | PK                                                                                             | 区域ID |
| Name        | NVARCHAR(100) | NOT NULL                                                                                       | 区域名称 |
| Code        | VARCHAR(20)   | NOT NULL, UNIQUE                                                                               | 区域编码 |
| Type        | VARCHAR(20)   | NOT NULL, DEFAULT 'public',<br>CHECK('public','academic','office','living','lab','restricted') | 区域类型 |
| AccessLevel | VARCHAR(20)   | NOT NULL, DEFAULT 'public',<br>CHECK('public','restricted','forbidden')                        | 开放   |

| 字段名         | 数据类型          | 约束                          | 说明   |
|-------------|---------------|-----------------------------|------|
|             |               |                             | 等级   |
| Description | NVARCHAR(500) | NULL                        | 区域描述 |
| IsActive    | BIT           | NOT NULL, DEFAULT 1         | 是否启用 |
| CreatedAt   | DATETIME2     | NOT NULL, DEFAULT GETDATE() | 创建时间 |

5.2.6 AreaPermissions（区域权限表）

| 字段名         | 数据类型        | 约束                                                                   | 说明      |
|-------------|-------------|----------------------------------------------------------------------|---------|
| Id          | INT         | PK                                                                   | 权限ID    |
| AreaId      | INT         | NOT NULL, FK → CampusAreas(Id) ON DELETE CASCADE                     | 区域ID    |
| VisitorType | VARCHAR(20) | NOT NULL, CHECK('parent','alumni','tourist','study_group','partner') | 允许的访客类型 |
| —           | —           | UNIQUE(AreaId, VisitorType)                                          | 复合唯一约束  |

5.2.7 Gates（校门表）

| 字段名      | 数据类型         | 约束                  | 说明   |
|----------|--------------|---------------------|------|
| Id       | INT          | PK                  | 校门ID |
| Name     | NVARCHAR(50) | NOT NULL            | 校门名称 |
| Code     | VARCHAR(20)  | NOT NULL, UNIQUE    | 校门编码 |
| IsActive | BIT          | NOT NULL, DEFAULT 1 | 是否启用 |

5.2.8 OpenRules（开放规则表）

| 字段名      | 数据类型        | 约束                                                             | 说明   |
|----------|-------------|----------------------------------------------------------------|------|
| Id       | INT         | PK                                                             | 规则ID |
| DateType | VARCHAR(20) | NOT NULL, CHECK('weekday','weekend','holiday','exam','custom') | 日期类型 |



| 字段名            | 数据类型          | 约束                                                        | 说明     |
|----------------|---------------|-----------------------------------------------------------|--------|
| StartDate      | DATE          | NOT NULL                                                  | 开始日期   |
| EndDate        | DATE          | NOT NULL                                                  | 结束日期   |
| TimeSlot       | VARCHAR(20)   | NOT NULL, <b>CHECK</b> ('morning','afternoon','full_day') | 开放时段   |
| MorningStart   | TIME          | NULL, DEFAULT '08:00'                                     | 上午开始   |
| MorningEnd     | TIME          | NULL, DEFAULT '12:00'                                     | 上午结束   |
| AfternoonStart | TIME          | NULL, DEFAULT '12:00'                                     | 下午开始   |
| AfternoonEnd   | TIME          | NULL, DEFAULT '17:00'                                     | 下午结束   |
| MaxCapacity    | INT           | NOT NULL, DEFAULT 500                                     | 最大接待人数 |
| IsActive       | BIT           | NOT NULL, DEFAULT 1                                       | 是否启用   |
| Remark         | NVARCHAR(500) | NULL                                                      | 备注     |

5.2.9 Activities（活动表）

| 字段名      | 数据类型          | 约束        | 说明   |
|----------|---------------|-----------|------|
| Id       | INT           | <b>PK</b> | 活动ID |
| Title    | NVARCHAR(200) | NOT NULL  | 活动标题 |
| Location | NVARCHAR(200) | NOT NULL  | 活动地点 |

| 字段名             | 数据类型           | 约束                                                                               | 说明     |
|-----------------|----------------|----------------------------------------------------------------------------------|--------|
| Description     | NVARCHAR(2000) | NULL                                                                             | 活动描述   |
| StartTime       | DATETIME2      | NOT NULL                                                                         | 开始时间   |
| EndTime         | DATETIME2      | NOT NULL                                                                         | 结束时间   |
| MaxParticipants | INT            | NOT NULL, DEFAULT 100                                                            | 人数上限   |
| CurrentCount    | INT            | NOT NULL, DEFAULT 0                                                              | 当前报名人数 |
| Status          | VARCHAR(20)    | NOT NULL, DEFAULT 'draft',<br><b>CHECK</b> ('draft','open','closed','cancelled') | 活动状态   |
| CoverImage      | VARCHAR(500)   | NULL                                                                             | 封面图URL |
| ContactPerson   | NVARCHAR(50)   | NULL                                                                             | 联系人    |
| ContactPhone    | VARCHAR(20)    | NULL                                                                             | 联系电话   |
| CreatedBy       | INT            | NOT NULL, <b>FK</b> → Users(Id)                                                  | 创建人    |

5.2.10 ActivityRegistrations（活动报名表）

| 字段名          | 数据类型         | 约束                                                                                               | 说明           |
|--------------|--------------|--------------------------------------------------------------------------------------------------|--------------|
| Id           | INT          | <b>PK</b>                                                                                        | 报名ID         |
| ActivityId   | INT          | NOT NULL, <b>FK</b> → Activities(Id)                                                             | 活动ID         |
| UserId       | INT          | NOT NULL, <b>FK</b> → Users(Id)                                                                  | 用户ID         |
| VisitorName  | NVARCHAR(50) | NOT NULL                                                                                         | 报名人姓名        |
| VisitorPhone | VARCHAR(20)  | NOT NULL                                                                                         | 报名人手机号       |
| Companions   | INT          | NOT NULL, DEFAULT 0                                                                              | 同行人数         |
| Status       | VARCHAR(20)  | NOT NULL, DEFAULT 'registered',<br><b>CHECK</b> ('registered','cancelled','checked_in','absent') | 报名状态         |
| CheckedInAt  | DATETIME2    | NULL                                                                                             | 签到时间         |
| —            | —            | <b>UNIQUE</b> (ActivityId, UserId)                                                               | 一人对一活动只能报名一次 |

5.2.11 EntryExitRecords（出入校记录表）

| 字段名           | 数据类型      | 约束                              | 说明    |
|---------------|-----------|---------------------------------|-------|
| Id            | INT       | PK                              | 记录ID  |
| ReservationId | INT       | NOT NULL, FK → Reservations(Id) | 关联预约  |
| UserId        | INT       | NOT NULL, FK → Users(Id)        | 用户ID  |
| EntryTime     | DATETIME2 | NULL                            | 入校时间  |
| EntryGateId   | INT       | NULL, FK → Gates(Id)            | 入校校门  |
| ExitTime      | DATETIME2 | NULL                            | 离校时间  |
| ExitGateId    | INT       | NULL, FK → Gates(Id)            | 离校校门  |
| OperatorId    | INT       | NULL, FK → Users(Id)            | 核验操作人 |

5.2.12 ViolationRecords（违规记录表）

| 字段名           | 数据类型           | 约束                                                                                               |
|---------------|----------------|--------------------------------------------------------------------------------------------------|
| Id            | INT            | PK                                                                                               |
| UserId        | INT            | NOT NULL, FK → Users(Id)                                                                         |
| ViolationType | VARCHAR(20)    | NOT NULL,<br>CHECK('no_show','overstay','trespass','duplicate','absence','disturbance','damage') |
| Description   | NVARCHAR(1000) | NOT NULL                                                                                         |
| OccurredAt    | DATETIME2      | NOT NULL                                                                                         |
| Location      | NVARCHAR(200)  | NULL                                                                                             |
| Severity      | VARCHAR(10)    | NOT NULL, DEFAULT 'minor', CHECK('minor','major','critical')                                     |
| SourceType    | VARCHAR(20)    | NOT NULL, DEFAULT 'system', CHECK('system','report','manual')                                    |

5.2.13 Blacklist（黑名单表）

| 字段名 | 数据类型 | 约束 | 说明   |
|-----|------|----|------|
| Id  | INT  | PK | 记录ID |

| 字段名            | 数据类型          | 约束                                             | 说明            |
|----------------|---------------|------------------------------------------------|---------------|
| UserId         | INT           | NOT NULL, <b>FK</b> → Users(Id), <b>UNIQUE</b> | 被拉黑用户（一人一条）   |
| Reason         | NVARCHAR(500) | NOT NULL                                       | 拉黑原因          |
| ViolationCount | INT           | NOT NULL, DEFAULT 0                            | 累计违规次数        |
| BlacklistedAt  | DATETIME2     | NOT NULL, DEFAULT GETDATE()                    | 拉黑时间          |
| ExpiresAt      | DATETIME2     | NULL                                           | 到期时间（NULL=永久） |
| IsActive       | BIT           | NOT NULL, DEFAULT 1                            | 是否生效          |
| OperatorId     | INT           | NOT NULL, <b>FK</b> → Users(Id)                | 操作人           |

5.2.14 Reports（举报表）

| 字段名           | 数据类型           | 约束                                                                               | 说明    |
|---------------|----------------|----------------------------------------------------------------------------------|-------|
| Id            | INT            | <b>PK</b>                                                                        | 举报ID  |
| ReporterId    | INT            | NOT NULL, <b>FK</b> → Users(Id)                                                  | 举报人   |
| TargetName    | NVARCHAR(100)  | NOT NULL                                                                         | 被举报对象 |
| ViolationType | VARCHAR(20)    | NOT NULL,<br><b>CHECK</b> ('trespass','disturbance','damage','overstay','other') | 违规类型  |
| Location      | NVARCHAR(200)  | NOT NULL                                                                         | 发生地点  |
| OccurredAt    | DATETIME2      | NOT NULL                                                                         | 发生时间  |
| Description   | NVARCHAR(2000) | NOT NULL                                                                         | 违规描述  |
| Status        | VARCHAR(20)    | NOT NULL, DEFAULT 'pending',<br><b>CHECK</b> ('pending','approved','rejected')   | 审核状态  |

5.2.15 StaffSchedules（排班表）

| 字段名     | 数据类型 | 约束                              | 说明     |
|---------|------|---------------------------------|--------|
| Id      | INT  | <b>PK</b>                       | 排班ID   |
| StaffId | INT  | NOT NULL, <b>FK</b> → Users(Id) | 工作人员ID |

| 字段名       | 数据类型          | 约束                                                                  | 说明             |
|-----------|---------------|---------------------------------------------------------------------|----------------|
| StaffRole | VARCHAR(20)   | NOT NULL, <b>CHECK</b> ('volunteer','guide','security')             | 角色             |
| WorkDate  | DATE          | NOT NULL                                                            | 工作日期           |
| Shift     | VARCHAR(20)   | NOT NULL, <b>CHECK</b> ('morning','afternoon','evening','full_day') | 班次             |
| —         | —             | <b>UNIQUE</b> (StaffId, WorkDate, Shift)                            | 同一人同一天同一班次不可重复 |
| StartTime | TIME          | NOT NULL                                                            | 开始时间           |
| EndTime   | TIME          | NOT NULL                                                            | 结束时间           |
| Location  | NVARCHAR(200) | NOT NULL                                                            | 工作地点           |
| Task      | NVARCHAR(500) | NULL                                                                | 工作内容           |

5.2.16 AuditLogs（审计日志表）

| 字段名          | 数据类型          | 约束                                                                                                            |
|--------------|---------------|---------------------------------------------------------------------------------------------------------------|
| Id           | BIGINT        | <b>PK</b> , IDENTITY(1,1)                                                                                     |
| OperatorId   | INT           | NULL, <b>FK</b> → Users(Id)                                                                                   |
| ActionType   | VARCHAR(30)   | NOT NULL, <b>CHECK</b> ('login','review','config','user_mgmt','data_export','blacklist','report','system','') |
| ActionDetail | NVARCHAR(500) | NOT NULL                                                                                                      |
| TargetType   | VARCHAR(50)   | NULL                                                                                                          |
| TargetId     | INT           | NULL                                                                                                          |
| IpAddress    | VARCHAR(50)   | NULL                                                                                                          |
| Result       | VARCHAR(10)   | NOT NULL, DEFAULT 'success', <b>CHECK</b> ('success','fail')                                                  |
| CreatedAt    | DATETIME2     | NOT NULL, DEFAULT GETDATE()                                                                                   |

5.3 物理结构设计

5.3.1 索引设计

为提高查询性能，在以下高频查询字段上建立了非聚集索引：

```
-- 预约表：按用户、状态、日期、手机号查询
CREATE NONCLUSTERED INDEX [IX_Reservations_UserId] ON Reservations(UserId)
CREATE NONCLUSTERED INDEX [IX_Reservations_Status] ON Reservations(Status)
CREATE NONCLUSTERED INDEX [IX_Reservations_VisitDate] ON Reservations(VisitDate)
CREATE NONCLUSTERED INDEX [IX_Reservations_VisitorPhone] ON Reservations(VisitorPhone)

-- 状态日志表：按预约ID追溯
CREATE NONCLUSTERED INDEX [IX_RSL_ReservationId] ON ReservationStatusLog(ReservationId)

-- 出入校记录：按用户、入校时间查询
CREATE NONCLUSTERED INDEX [IX_EER_UserId] ON EntryExitRecords(UserId)
CREATE NONCLUSTERED INDEX [IX_EER_EntryTime] ON EntryExitRecords(EntryTime)

-- 活动报名：按活动ID统计
CREATE NONCLUSTERED INDEX [IX_AR_ActivityId] ON ActivityRegistrations(ActivityId)

-- 违规记录：按用户查询违规历史
```

```

CREATE NONCLUSTERED INDEX [IX_VR_UserId] ON ViolationRecords(UserId)

-- 审计日志：复合查询优化
CREATE NONCLUSTERED INDEX [IX_AL_OperatorId] ON AuditLogs(OperatorId)
CREATE NONCLUSTERED INDEX [IX_AL_CreatedAt] ON AuditLogs(CreatedAt)
CREATE NONCLUSTERED INDEX [IX_AL_ActionType] ON AuditLogs(ActionType)

-- 举报：按状态筛选
CREATE NONCLUSTERED INDEX [IX_Rep_Status] ON Reports(Status)

-- 排班：按日期查询
CREATE NONCLUSTERED INDEX [IX_SS_WorkDate] ON StaffSchedules(WorkDate)

```

#### 索引设计理由：

- **Reservations** 是核心查询表，按 **Status**（审核列表）、**VisitDate**（当日预约）、**VisitorPhone**（安保核验）是最高频的查询条件
- **AuditLogs** 表数据量会随时间线性增长，必须对 **CreatedAt** 和 **ActionType** 建立索引以支持按时间和类型筛选
- **EntryExitRecords** 查询"当前在校访客"需要 **EntryTime IS NOT NULL AND ExitTime IS NULL**，对 **EntryTime** 建立索引可加速该过滤

### 5.3.2 触发器设计

**核心触发器：** **trg\_Reservations\_StatusChange** — 预约状态变更自动记录日志

```

CREATE TRIGGER [dbo].[trg_Reservations_StatusChange]
ON [dbo].[Reservations]
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON
    -- 仅当 Status 字段被更新时触发
    IF UPDATE([Status])
    BEGIN
        INSERT INTO [dbo].[ReservationStatusLog]
            ([ReservationId], [FromStatus], [ToStatus], [OperatorId], [Remark])
        SELECT
            i.[Id], -- 预约ID
            d.[Status], -- 变更前状态（来自 DELETED 伪表）
            i.[Status], -- 变更后状态（来自 INSERTED 伪表）
            i.[ReviewerId], -- 操作人
            CASE i.[Status]
                WHEN 'approved' THEN N'审核通过'
                WHEN 'rejected' THEN N'审核拒绝: ' + ISNULL(i.[ReviewRemark], N'')
                WHEN 'cancelled' THEN N'用户取消预约'
                WHEN 'checked_in' THEN N'已入校核验'
                WHEN 'checked_out' THEN N'已离校登记'
                WHEN 'no_show' THEN N'系统标记爽约'
                ELSE N'状态变更'
            END
        FROM INSERTED i
        INNER JOIN DELETED d ON i.[Id] = d.[Id]
    END
END
GO

```

#### 触发器设计说明：

- 利用 SQL Server 的 **INSERTED** 和 **DELETED** 伪表对比获取状态变更前后的值
- 使用 **IF UPDATE(Status)** 条件判断，仅在 Status 字段变更时才执行插入，避免不必要的日志写入
- 自动根据目标状态生成中文备注，减少应用层代码负担
- 这意味着**任何途径**（无论是前端 API 调用还是直接 SQL 操作）修改预约状态，都会自动记录日志，保证审计完整性

### 5.3.3 自定义函数设计

**预约编号生成函数：**

```

CREATE FUNCTION [dbo].[GenerateReservationNo]()
RETURNS VARCHAR(30)

```

```
AS
BEGIN
    DECLARE @dateStr VARCHAR(10) = FORMAT(GETDATE(), 'yyyyMMdd')
    DECLARE @seq INT

    -- 查询当日最大序号并 +1
    SELECT @seq = ISNULL(MAX(CAST(RIGHT([ReservationNo], 4) AS INT)), 0) + 1
    FROM [dbo].[Reservations]
    WHERE [ReservationNo] LIKE 'R' + @dateStr + '%'

    -- 返回格式: R + 日期 + 4位序号, 如 R202606260001
    RETURN 'R' + @dateStr + RIGHT('0000' + CAST(@seq AS VARCHAR), 4)
END
GO
```

设计要点：

- 预约编号格式：R + 8位日期 + 4位流水号（如 R202606260001）
- 按日期独立编号，每天从 0001 开始
- 使用 ISNULL(MAX(...), 0) + 1 确保即使当日无预约也能正确从 1 开始
- 配合 ReservationNo 字段的 UNIQUE 约束，保证编号全局唯一

5.3.4 约束汇总

| 约束类型           | 数量  | 示例                                                                   |
|----------------|-----|----------------------------------------------------------------------|
| PRIMARY KEY    | 16  | 每表一个自增主键                                                             |
| FOREIGN KEY    | 25+ | Reservations.UserId → Users.Id 等                                     |
| UNIQUE         | 8   | Users.Phone、Reservations.ReservationNo、Blacklist.UserId 等            |
| CHECK          | 15+ | Users.Role IN (...)、Reservations.Status IN (...) 等                   |
| DEFAULT        | 20+ | Users.Role DEFAULT 'visitor'、Reservations.Status DEFAULT 'pending' 等 |
| CASCADE DELETE | 1   | AreaPermissions.AreaId → CampusAreas.Id（删除区域时级联删除权限）                 |

关键约束设计说明：

1. CHECK 约束的枚举值控制：所有状态字段（Role、Status、VisitorType、TimeSlot 等）都使用 CHECK 约束限制取值，确保数据层面不会出现非法状态值，比应用层校验更可靠
2. ON DELETE NO ACTION：大部分外键使用 NO ACTION（默认行为），防止误删导致级联数据丢失，例如删除用户时不允许直接删除（因为有关联的预约记录），需要业务层先处理
3. ON DELETE CASCADE：仅在 AreaPermissions 上使用，因为权限是区域的附属属性，删除区域时权限也失去意义

6. 后端设计

6.1 项目结构

```
backend/
├── Controllers/           # 控制层 - 处理HTTP请求
│   ├── AuthController.cs # 认证（登录/注册/修改密码）
│   ├── ReservationController.cs # 预约管理（创建/查询/审核/取消）
│   ├── ActivityController.cs # 活动管理（发布/报名/签到）
│   ├── EntryExitController.cs # 出入校管理（核验/入校/离校）
│   ├── AdminController.cs # 后台管理（看板/规则/黑名单/排班）
│   ├── ReportController.cs # 举报管理
│   └── PublicController.cs # 公开接口（校园状态等）
├── Services/             # 服务层 - 核心业务逻辑
│   ├── IAuthService.cs / AuthService.cs
│   ├── IReservationService.cs / ReservationService.cs
│   └── IActivityService.cs / ActivityService.cs
```

```
| | IEntryExitService.cs / EntryExitService.cs
| | IAdminService.cs / AdminService.cs
| Models/ # 数据模型 - EF Core实体映射
| | User.cs, Visitor.cs, Reservation.cs
| | CampusArea.cs, AreaPermission.cs, Gate.cs, OpenRule.cs
| | Activity.cs, ActivityRegistration.cs
| | EntryExitRecord.cs, ReservationStatusLog.cs
| | ViolationRecord.cs, Blacklist.cs
| | Report.cs, StaffSchedule.cs, AuditLog.cs
| DTOs/ # 数据传输对象
| | AuthDtos.cs, ReservationDtos.cs, ActivityDtos.cs
| | EntryExitDtos.cs, AdminDtos.cs, ReportDtos.cs
| Data/
| | CampusVisitorDbContext.cs # 数据库上下文
| Mappings/
| | MappingProfile.cs # AutoMapper配置
| Program.cs # 应用入口 + 依赖注入配置
| appsettings.json # 配置文件（连接字符串、JWT密钥）
```

## 6.2 认证与授权机制

系统使用 JWT（JSON Web Token）实现无状态认证，结合 基于角色的访问控制（RBAC）：

```
// Program.cs 中的配置
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => {
        // Token 验证参数: 签发者、受众、生命周期、签名密钥
        options.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = "CampusVisitorApi",
            ValidAudience = "CampusVisitorApp",
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(jwtKey))
        };
    });
```

Token 中包含的 Claims：

- **NameIdentifier**：用户ID
- **Name**：用户姓名
- **MobilePhone**：手机号
- **Role**：角色（visitor/admin/security/staff）

Controller 级别的授权控制：

| 接口                         | 授权要求                            |
|----------------------------|---------------------------------|
| POST /api/auth/login       | 无（公开）                           |
| POST /api/reservations     | [Authorize]（任意登录用户）             |
| GET /api/reservations      | [Authorize(Roles = "admin")]    |
| POST /api/entry-exit/entry | [Authorize(Roles = "security")] |
| GET /api/admin/dashboard   | [Authorize(Roles = "admin")]    |

## 6.3 核心业务逻辑详解

### 6.3.1 预约创建流程

```
// ReservationService.CreateAsync()
public async Task CreateAsync(int userId, CreateReservationRequest request)
{
    // 1. 查询当日最后一条预约编号，生成新的流水号
    var last = await _db.Reservations
        .OrderByDescending(r => r.Id)
```



```

        .Select(r => r.ReservationNo)
        .FirstOrDefaultAsync();

var seq = 1;
if (last != null) seq = int.Parse(last[^4..]) + 1;

// 2. 创建 Reservation 实体
var reservation = new Reservation
{
    UserId = userId,
    ReservationNo = $"R{DateTime.Now:yyyyMMdd}{seq:D4}", // 如 R202606260001
    VisitorType = request.VisitorType,
    VisitorName = request.Name,
    VisitorPhone = request.Phone,
    VisitDate = request.VisitDate,
    TimeSlot = request.TimeSlot,
    Companions = request.Companions,
    StayDuration = request.StayDuration,
    Purpose = request.Purpose,
    Status = "pending", // 初始状态: 待审核
};

// 3. 保存到数据库 (触发器自动写入 ReservationStatusLog)
_db.Reservations.Add(reservation);
await _db.SaveChangesAsync();
}

```

**数据库交互:** INSERT INTO Reservations → 触发器 trg\_Reservations\_StatusChange 自动插入 ReservationStatusLog (FromStatus=NULL, ToStatus='pending')

### 6.3.2 预约审核流程

```

// ReservationService.ReviewAsync()
public async Task ReviewAsync(int reviewerId, int id, ReviewReservationRequest request)
{
    var reservation = await _db.Reservations.FindAsync(id)
        ?? throw new KeyNotFoundException("预约不存在");

    if (reservation.Status != "pending")
        throw new InvalidOperationException("只能审核待处理状态的预约");

    reservation.Status = request.Action; // "approved" 或 "rejected"
    reservation.ReviewerId = reviewerId;
    reservation.ReviewRemark = request.Remark;
    reservation.ReviewedAt = DateTime.UtcNow;

    // 保存后触发器自动写入状态变更日志
    await _db.SaveChangesAsync();
}

```

**数据库交互:** UPDATE Reservations SET Status='approved', ReviewerId=..., ... → 触发器检测 Status 变更 → INSERT INTO ReservationStatusLog(FromStatus='pending', ToStatus='approved', ...)

### 6.3.3 入校核验流程

```

// EntryExitService.EntryAsync()
public async Task<EntryExitRecord> EntryAsync(int operatorId, EntryCheckRequest request)
{
    var reservation = await _db.Reservations.FindAsync(request.Id)
        ?? throw new KeyNotFoundException("预约不存在");

    // 业务规则: 只有审核通过的预约才能入校
    if (reservation.Status != "approved")
        throw new InvalidOperationException("该预约未通过审核, 不可入校");

    // 查找对应的校门
    var gate = await _db.Gates.FirstOrDefaultAsync(g => g.Name == request.Gate)
        ?? await _db.Gates.FirstAsync();

    // 更新预约状态
    reservation.Status = "checked_in";

    // 创建出入校记录
    var record = new EntryExitRecord
    {

```

```

        ReservationId = reservation.Id,
        UserId = reservation.UserId,
        EntryTime = DateTime.UtcNow,
        EntryGateId = gate.Id,
        OperatorId = operatorId,
    };

    _db.EntryExitRecords.Add(record);
    await _db.SaveChangesAsync(); // 一个事务完成两个操作
    return record;
}

```

**数据库交互：** `UPDATE Reservations SET Status='checked_in' + INSERT INTO EntryExitRecords(...)` 在同一事务中执行 → 触发器记录状态变更

### 6.3.4 活动报名 — 并发控制

```

public async Task RegisterAsync(int userId, RegisterActivityRequest request)
{
    // 1. 检查活动是否存在
    var activity = await _db.Activities.FindAsync(request.ActivityId)
    ?? throw new KeyNotFoundException("活动不存在");

    // 2. 检查名额（业务层控制，配合数据库 UNIQUE 约束兜底）
    if (activity.CurrentCount >= activity.MaxParticipants)
        throw new InvalidOperationException("报名人数已满");

    // 3. 检查是否重复报名
    if (await _db.ActivityRegistrations.AnyAsync(r =>
        r.ActivityId == request.ActivityId && r.UserId == userId))
        throw new InvalidOperationException("您已报名该活动");

    // 4. 创建报名记录 + 更新活动计数
    var registration = new ActivityRegistration { ... };
    activity.CurrentCount++; // 计数 +1

    _db.ActivityRegistrations.Add(registration);
    await _db.SaveChangesAsync(); // 同一事务，保证计数准确性
}

```

**并发安全设计：**

- 业务层校验 `CurrentCount >= MaxParticipants`（乐观检查）
- 数据库层面 `UNIQUE(ActivityId, UserId)` 约束防止重复报名（兜底保障）
- EF Core 的 `SaveChangesAsync()` 在同一个数据库事务中完成"插入报名记录"和"更新活动计数"两个操作

## 7. 前端设计

### 7.1 前端项目结构

```

frontend/
├─ index.html
├─ package.json
├─ vite.config.js          # Vite 构建配置（含 API 代理）
├─ src/
│   ├─ main.js             # 应用入口
│   ├─ App.vue             # 根组件
│   └─ api/                # API 请求层
│       ├─ request.js      # Axios 实例（拦截器配置）
│       ├─ auth.js         # 认证 API
│       ├─ reservation.js  # 预约 API
│       ├─ activity.js     # 活动 API
│       ├─ entryexit.js    # 出入校 API
│       ├─ admin.js        # 管理 API
│       └─ report.js       # 举报 API
│   └─ router/             # 路由配置（含角色守卫）
│       └─ index.js
│   └─ stores/             # Pinia 认证状态管理
│       └─ auth.js

```

|                            |          |
|----------------------------|----------|
| components/                | # 布局组件   |
| ├ VisitorLayout.vue        | # 访客端布局  |
| ├ AdminLayout.vue          | # 管理员端布局 |
| └ SecurityLayout.vue       | # 安保端布局  |
| views/                     | # 页面视图   |
| ├ visitor/                 |          |
| │ └ Reservation.vue        | # 入校预约页  |
| │ └ ActivityList.vue       | # 校园活动浏览 |
| │ └ MyReservations.vue     | # 我的预约   |
| │ └ VisitorProfile.vue     | # 个人中心   |
| ├ admin/                   |          |
| │ └ Dashboard.vue          | # 数据看板   |
| │ └ ReservationReview.vue  | # 预约审核   |
| │ └ OpenRules.vue          | # 开放规则管理 |
| │ └ AreaManagement.vue     | # 区域管理   |
| │ └ ActivityManagement.vue | # 活动管理   |
| │ └ Blacklist.vue          | # 黑名单管理  |
| │ └ StaffsSchedule.vue     | # 排班管理   |
| │ └ AuditLog.vue           | # 审计日志   |
| │ └ ReportManagement.vue   | # 举报管理   |
| ├ security/                |          |
| │ └ EntryCheck.vue         | # 入校核验   |
| │ └ ExitRecord.vue         | # 离校登记   |
| │ └ CurrentVisitors.vue    | # 当前在校访客 |
| │ └ SecurityReport.vue     | # 安保报告   |
| └ auth/                    | # 登录/注册页 |

## 7.2 路由设计与角色守卫

```
// 访问控制：
// 1. 访客端 (/visitor/*) - 公开浏览，但"我的预约"和"个人中心"需登录
// 2. 管理员端 (/admin/*) - 必须登录 且 角色为 admin
// 3. 安保端 (/security/*) - 必须登录 且 角色为 security

// 路由守卫核心逻辑
router.beforeEach((to, from, next) => {
  if (to.meta.requiresAuth && !authStore.isLoggedIn) {
    next('/auth/login') // 未登录 → 跳转登录页
  } else if (to.meta.role && authStore.user?.role !== to.meta.role) {
    next('/') // 角色不匹配 → 跳转首页
  } else {
    next()
  }
})
```

## 7.3 前后端通信

前端通过 Axios 实例统一管理 HTTP 请求，核心配置：

```
// request.js - Axios 实例
const api = axios.create({
  baseURL: '/api', // 通过 Vite 代理转发到后端 http://localhost:5000
  timeout: 10000,
})

// 请求拦截器：自动附加 JWT Token
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token')
  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})

// 响应拦截器：统一错误处理
api.interceptors.response.use(
  response => response.data, // 直接返回 data
  error => {
    if (error.response?.status === 401) {
      // Token 过期 → 清除登录状态 → 跳转登录页
    }
    return Promise.reject(error)
  }
)
```

---

## 8. 成果展示与功能演示

---

### 8.1 访客端功能

#### 功能 F1：用户注册与登录

- 页面：/auth/login
- 对应需求：REQ-01
- 功能描述：访客通过手机号注册账号（密码 BCrypt 加密存储），已有账号直接登录；管理员和安保使用预置账号登录
- 前端实现：Element Plus 表单组件，手机号正则校验，登录成功后将 JWT Token 存入 localStorage
- 后端实现：AuthController.Login() → AuthService.LoginAsync() → 验证密码 → 生成 JWT Token
- 数据库操作：SELECT \* FROM Users WHERE Phone = ? → 验证 PasswordHash → UPDATE Users SET LastLoginAt = GETDATE()

#### 功能 F2：入校预约提交

- 页面：/visitor/reservation
- 对应需求：REQ-02
- 功能描述：访客填写姓名、手机号、选择访客类型（家长/校友/游客/研学团/合作单位）、选择入校日期和时段（上午/下午/全天）、填写同行人数和入校事由，提交预约申请
- 前端实现：
  - 日期选择器禁用过去日期（disabledDate 函数）
  - 表单校验：姓名必填、手机号正则 ^1[3-9]\d{9}\$、访客类型/日期/时段/事由必填
  - 右侧展示"预约须知"和"今日校园状态"（从 GET /api/public/campus-status 获取实时数据）
- 后端实现：ReservationController.Create() → ReservationService.CreateAsync() → 生成预约编号 → 插入数据库
- 数据库操作：
  - INSERT INTO Reservations (UserId, ReservationNo, VisitorType, VisitorName, ...) VALUES (...)
  - 触发器自动 INSERT INTO ReservationStatusLog (FromStatus=NULL, ToStatus='pending')

#### 功能 F3：我的预约管理

- 页面：/visitor/my-reservations
- 对应需求：REQ-02
- 功能描述：访客查看自己的预约列表，支持按状态筛选（待审核/已通过/已拒绝/已入校/已离校），可取消"待审核"状态的预约
- 后端实现：ReservationController.GetMy() → ReservationService.GetMyReservationsAsync() → 分页查询
- 数据库操作：SELECT \* FROM Reservations WHERE UserId = ? AND Status = ? ORDER BY CreatedAt DESC OFFSET ? ROWS FETCH NEXT ? ROWS ONLY

#### 功能 F4：校园活动浏览与报名

- 页面：/visitor/activities
- 对应需求：REQ-06
- 功能描述：访客浏览已开放的活动列表，查看活动详情（标题、地点、时间、人数上限/当前报名人数），点击报名
- 前端实现：卡片列表展示活动，报名按钮在名额已满时禁用
- 后端实现：ActivityController.Register() → ActivityService.RegisterAsync() → 名额检查和去重
- 数据库操作：

- `SELECT * FROM Activities WHERE Status = 'open'` (活动列表)
- `INSERT INTO ActivityRegistrations + UPDATE Activities SET CurrentCount = CurrentCount + 1` (报名事务)

## 8.2 管理员端功能

### 功能 F5：数据看板

- 页面：`/admin/dashboard`
- 对应需求：REQ-13
- 功能描述：展示今日预约人数、当前在校访客数、待审核预约数、黑名单人数等核心统计指标，以及客流趋势和访客类型分布图表
- 前端实现：统计卡片 + 图表占位区（可集成 ECharts），数据从 `GET /api/admin/dashboard` 获取
- 后端实现：`AdminController.GetDashboard()` → `AdminService.GetDashboardStatsAsync()` → 聚合查询
- 数据库操作：
  - `SELECT COUNT(*) FROM Reservations WHERE VisitDate = CAST(GETDATE() AS DATE)`
  - `SELECT COUNT(*) FROM EntryExitRecords WHERE EntryTime IS NOT NULL AND ExitTime IS NULL`
  - `SELECT COUNT(*) FROM Reservations WHERE Status = 'pending'`
  - `SELECT COUNT(*) FROM Blacklist WHERE IsActive = 1`

### 功能 F6：预约审核

- 页面：`/admin/review`
- 对应需求：REQ-03
- 功能描述：管理员查看待审核预约列表，点击查看详情，选择"通过"或"拒绝"并填写审核意见
- 前端实现：Table 表格展示预约列表，状态标签颜色区分（warning=待审核, success=已通过, danger=已拒绝），审核对话框
- 后端实现：`ReservationController.Review()` → `ReservationService.ReviewAsync()` → 更新状态和审核信息
- 数据库操作：
  - `SELECT * FROM Reservations WHERE Status = 'pending' ORDER BY CreatedAt DESC` (列表)
  - `UPDATE Reservations SET Status = 'approved', ReviewerId = ?, ReviewRemark = ?, ReviewedAt = GETDATE()` (审核操作)
  - 触发器：`INSERT INTO ReservationStatusLog (FromStatus='pending', ToStatus='approved', ...)`

### 功能 F7：开放规则管理

- 页面：`/admin/open-rules`
- 对应需求：REQ-08
- 功能描述：管理员配置不同日期类型（工作日/周末/节假日/考试周/自定义）的开放时段、容量上限，支持启用/禁用
- 数据库操作：`INSERT/UPDATE/DELETE` 操作 `OpenRules` 表

### 功能 F8：区域权限管理

- 页面：`/admin/areas`
- 对应需求：REQ-09
- 功能描述：管理员定义校园区域（名称、编码、类型、开放等级），配置每种访客类型对该区域的访问权限
- 数据库操作：
  - `CampusAreas` 表（区域基本信息）
  - `AreaPermissions` 表（多对多关联：区域 ↔ 访客类型）
  - 使用 `ON DELETE CASCADE`，删除区域自动清除相关权限记录

#### 功能 F9：黑名单管理

- 页面：/admin/blacklist
- 对应需求：REQ-10
- 功能描述：管理员查看黑名单列表，可将违规访客移出黑名单
- 数据库操作：UPDATE Blacklist SET IsActive = 0, UnblacklistedAt = GETDATE()

#### 功能 F10：审计日志

- 页面：/admin/audit-logs
- 对应需求：REQ-14
- 功能描述：管理员查看所有后台操作的审计日志，支持按操作类型筛选
- 数据库操作：SELECT \* FROM AuditLogs WHERE ActionType = ? ORDER BY CreatedAt DESC

## 8.3 安保端功能

#### 功能 F11：入校核验

- 页面：/security/entry-check
- 对应需求：REQ-04
- 功能描述：安保人员通过预约编号/手机号/身份证号查询预约信息，确认后点击"确认入校"，记录入校时间和校门
- 前端实现：搜索框 + 核验结果展示（Element Plus Descriptions 组件），绿色确认按钮仅对 status=approved 的预约可用
- 后端实现：EntryExitController.Search() → SearchAsync()（查询）→ EntryExitController.Entry() → EntryAsync()（入校登记）
- 数据库操作：
  - 查询：SELECT \* FROM Reservations WHERE ReservationNo = ? OR VisitorPhone = ? OR VisitorName LIKE '%?%'
  - 入校：UPDATE Reservations SET Status = 'checked\_in' + INSERT INTO EntryExitRecords (EntryTime, EntryGateId, OperatorId, ...)
  - 触发器自动记录状态变更

#### 功能 F12：离校登记

- 页面：/security/exit-record
- 对应需求：REQ-05
- 功能描述：安保人员输入访客姓名，查询在记录中尚未离校的访客，登记离校时间和校门
- 后端实现：EntryExitController.Exit() → ExitAsync() → 查找未离校记录 → 更新离校信息
- 数据库操作：
  - SELECT \* FROM EntryExitRecords WHERE Reservation.VisitorName = ? AND ExitTime IS NULL
  - UPDATE EntryExitRecords SET ExitTime = GETDATE(), ExitGateId = ? + UPDATE Reservations SET Status = 'checked\_out'

#### 功能 F13：当前在校访客

- 页面：/security/current-visitors
- 对应需求：REQ-04
- 功能描述：展示当前在校访客列表（已入校但未离校），方便安保掌握校园内访客情况
- 数据库操作：SELECT \* FROM EntryExitRecords WHERE EntryTime IS NOT NULL AND ExitTime IS NULL

---

## 9. 功能与需求对照表

---

| 需求编号   | 需求描述    | 实现功能                       | 前端页面                                           | 后端接口                                                    | 核心数据库表                                |
|--------|---------|----------------------------|------------------------------------------------|---------------------------------------------------------|---------------------------------------|
| REQ-01 | 用户注册/登录 | F1: 手机号注册登录, JWT 认证        | 登录/注册页                                         | POST /api/auth/login, /register                         | Users                                 |
| REQ-02 | 入校预约    | F2: 提交预约;<br>F3: 我的预约      | /visitor/reservation, /visitor/my-reservations | POST /api/reservations, GET /api/reservations/my        | Reservations                          |
| REQ-03 | 预约审核    | F6: 管理员审核预约                | /admin/review                                  | PUT /api/reservations/{id}/review                       | Reservations, ReservationStatusLog    |
| REQ-04 | 入校核验    | F11: 安保查询并确认入校             | /security/entry-check                          | POST /api/entry-exit/search, /entry                     | Reservations, EntryExitRecords, Gates |
| REQ-05 | 离校登记    | F12: 安保登记离校                | /security/exit-record                          | PUT /api/entry-exit/exit                                | EntryExitRecords, Reservations        |
| REQ-06 | 活动管理    | F4: 访客浏览报名;<br>F5: 管理员管理活动 | /visitor/activities, /admin/activities         | GET/POST /api/activities, POST /api/activities/register | Activities, ActivityRegistrations     |
| REQ-07 | 活动签到    | F5: 管理员现场签到                | /admin/activities                              | PUT /api/activities/checkin/{id}                        | ActivityRegistrations                 |
| REQ-08 | 开放规则    | F7: 配置开放日期/                | /admin/open-rules                              | GET/POST/DELETE /api/open-rules                         | OpenRules                             |

| 需求编号   | 需求描述  | 实现功能          | 前端页面                           | 后端接口                                        | 核心数据库表                       |
|--------|-------|---------------|--------------------------------|---------------------------------------------|------------------------------|
|        |       | 时段/容量         |                                |                                             |                              |
| REQ-09 | 区域权限  | F8: 定义区域和访问权限 | <code>/admin/areas</code>      | <code>GET/POST/DELETE /api/areas</code>     | CampusAreas, AreaPermissions |
| REQ-10 | 黑名单管理 | F9: 查看和移出黑名单  | <code>/admin/blacklist</code>  | <code>GET/DELETE /api/blacklist</code>      | Blacklist                    |
| REQ-11 | 违规举报  | 举报提交与管理       | 举报相关页面                         | <code>ReportController</code>               | Reports                      |
| REQ-12 | 排班管理  | 安保/工作人员排班     | <code>/admin/schedules</code>  | <code>GET/POST/DELETE /api/schedules</code> | StaffSchedules               |
| REQ-13 | 数据看板  | F5: 核心统计展示    | <code>/admin/dashboard</code>  | <code>GET /api/admin/dashboard</code>       | 多表聚合查询                       |
| REQ-14 | 审计日志  | F10: 操作日志查看   | <code>/admin/audit-logs</code> | <code>GET /api/audit-logs</code>            | AuditLogs                    |
| REQ-15 | 状态追溯  | 自动记录（触发器）     | 预约详情页可查看                       | 预约详情接口含状态日志                                 | ReservationStatusLog（触发器写入）  |

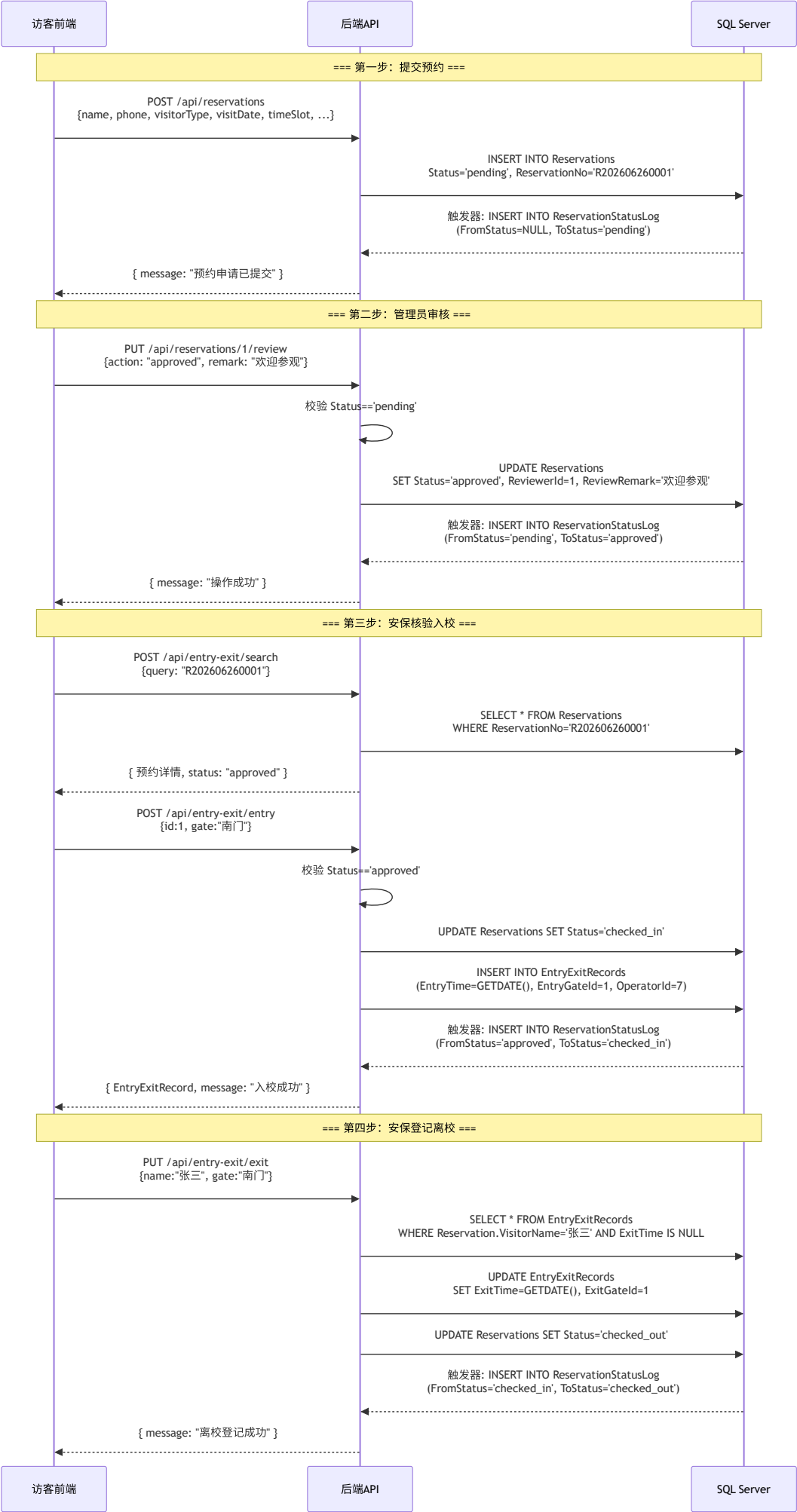
## 10. 前端-后端-数据库关联分析

本章选取系统中最具代表性的三个**关联业务行为**，详细分析从前端用户操作到后端业务逻辑再到数据库持久化的完整数据流。

### 10.1 业务链路一：访客预约 → 审核 → 入校 → 离校（全流程）

这是系统的**核心业务流程**，涉及 4 个角色（访客、管理员、安保 ×2 次）、4 个数据库表、1 个触发器和 5 次状态变更。

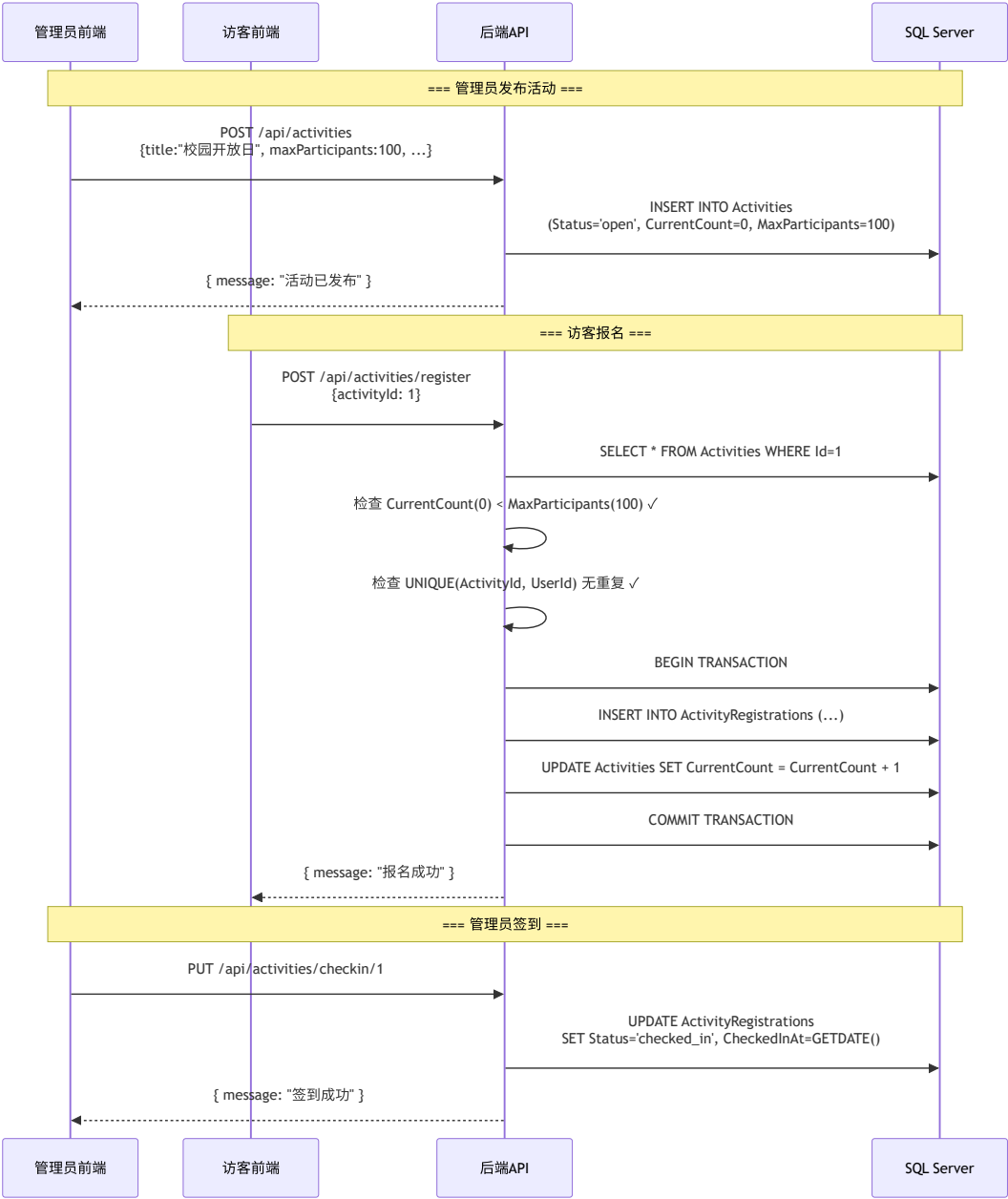




数据库关联分析：

| 步骤   | 涉及表                                                                            | 关联方式                                                                               | 数据一致性保障                                     |
|------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------|---------------------------------------------|
| 预约提交 | <code>Reservations</code> + <code>ReservationStatusLog</code>                  | <code>ReservationId</code> 外键                                                      | 触发器自动记录，确保不遗漏                               |
| 审核   | <code>Reservations</code> + <code>Users</code><br>(审核人)                        | <code>ReviewerId</code> → <code>Users.Id</code>                                    | 外键约束 + 状态 CHECK 约束                          |
| 入校   | <code>Reservations</code> + <code>EntryExitRecords</code> + <code>Gates</code> | <code>ReservationId</code> ,<br><code>EntryGateId</code> , <code>OperatorId</code> | 同一事务中完成状态更新和记录插入                            |
| 离校   | <code>EntryExitRecords</code> + <code>Reservations</code>                      | <code>ReservationId</code>                                                         | 通过 <code>ExitTime IS NULL</code> 条件确保只更新在记录 |

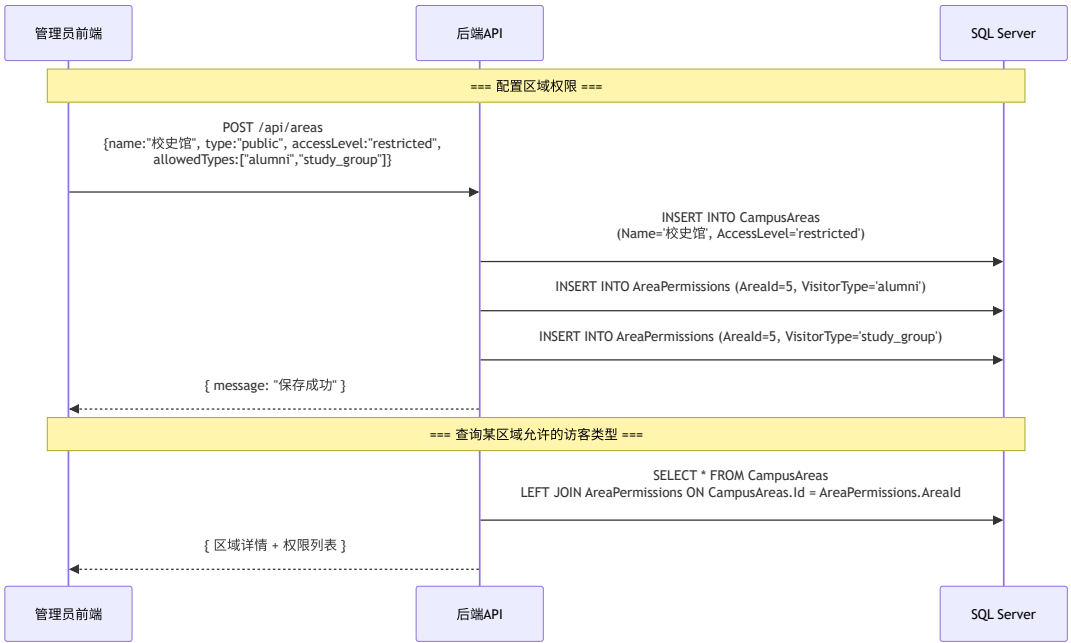
## 10.2 业务链路二：活动发布 → 报名 → 签到（并发控制）



关键数据库设计：

- `Activities.CurrentCount` 是冗余字段（可从 `ActivityRegistrations` 聚合计算），但保留它通过应用层同步更新，避免每次查询活动列表时都要 `COUNT` 子查询
- `UNIQUE(ActivityId, UserId)` 约束在数据库层面兜底防止重复报名
- 报名操作在**同一事务**中完成"插入报名记录"和"递增计数"，保证数据一致性

## 10.3 业务链路三：区域权限管控



多对多关系设计说明：

- `CampusAreas` 和访客类型之间的多对多关系通过 `AreaPermissions` 中间表实现
- `UNIQUE(AreaId, VisitorType)` 约束确保同一区域对同一访客类型不会出现重复权限记录
- `ON DELETE CASCADE`：删除区域时自动清除相关联的权限记录，防止孤儿数据
- 这种设计使系统具备良好的扩展性——新增访客类型或区域时无需修改表结构，只需增删 `AreaPermissions` 记录

## 11. 总结与展望

### 11.1 项目完成情况总结

本项目完整实现了一个校园访客管理系统的前后端全栈开发，核心成果包括：

| 维度  | 完成内容                                                                                               |
|-----|----------------------------------------------------------------------------------------------------|
| 数据库 | 设计并实现 16 张表，包含完整的 ER 关系、25+ 外键约束、15+ CHECK 约束、8 个 UNIQUE 约束、13 个非聚集索引、1 个触发器、1 个自定义函数              |
| 后端  | 基于 ASP.NET Core Web API 实现 7 个 Controller、5 个 Service、15 个 Model，使用 JWT + RBAC 认证授权，EF Core 作为 ORM |
| 前端  | 基于 Vue 3 + Element Plus 实现三套界面（访客端、管理员端、安保端），25+ 页面，Axios 统一请求管理，路由守卫实现页面级权限控制                     |
| 业务  | 覆盖预约全生命周期（提交→审核→入校→离校）、活动管理、区域权限、黑名单、排班、举报、审计等 15 项功能需求                                            |

### 11.2 技术亮点

1. **触发器实现无侵入审计**：预约状态变更日志完全由数据库触发器自动记录，应用层无需关心，任何途径修改状态都能被追溯
2. **CHECK 约束的枚举值控制**：所有状态字段都在数据库层面通过 CHECK 约束限制合法取值，比纯应用层校验更可靠
3. **多对多权限模型**：`AreaPermissions` 中间表实现区域与访客类型的灵活权限配置
4. **前后端分离 + JWT 认证**：符合现代 Web 开发最佳实践，前后端独立部署、独立开发
5. **分层架构**：Controller → Service → EF Core → Database，职责清晰，便于测试和维护

# 11.3 不足与改进方向

| 不足                  | 改进方向                                                   |
|---------------------|--------------------------------------------------------|
| 活动报名在高并发下可能出现超卖     | 使用数据库行锁（ <code>SELECT ... FOR UPDATE</code> ）或乐观锁（版本号） |
| 预约编号由应用层生成，存在并发冲突风险 | 改用数据库 SEQUENCE 或 <code>GenerateReservationNo</code> 函数 |
| 缺少数据可视化图表           | 集成 ECharts 实现客流趋势图和访客类型饼图                              |
| 缺少文件上传功能            | 活动封面图、举报证据等目前仅支持 URL，可扩展为文件上传                          |
| 缺少消息通知              | 预约审核结果可通过短信或站内信通知访客                                    |

项目仓库：<https://github.com/eiyovai/database>

技术栈：SQL Server 2019 + ASP.NET Core 8.0 + Vue 3 + Element Plus + Vite